Evaluation Report: Building Machine Learning Pipelines on the Urban Data Integration Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

October 4, 2026

1 Evaluation Method

The model predicts the base fare of one taxi trip. We built the same training dataset in two ways. **Approach A** starts from the raw files in `data/raw/` and re-implements every platform rule that affects the training rows. **Approach B** starts from the Delta table `integrated_taxi_trips` built in Weeks 1–3. From feature engineering onward, both approaches run the same Task 2 and Task 3 notebooks, which import one shared definition from `src/data_intensive_computing/week4/`. Any difference between the approaches therefore comes from data preparation.

All measurements ran on one MacBook Pro (Apple M5 Pro, 48 GB RAM) with Spark 4.2 and Delta Lake 4.4. Every notebook ran with `jupyter nbconvert` in a fresh kernel, round-robin with a rotating start position. We discarded one warm-up round and report the median of five measured rounds (three for the slower workflows). The harness is `src/data_intensive_computing/week4/timing.py`; raw results are in `artifacts/task4_timing_results.jsonl`.

Table 1: Measured workflows

Workflow	What it runs	Spark settings
Spark baseline	Spark start-up only	`local[*]`, 8 GB, 32 partitions
A dataset	Raw files → training dataset	`local[*]`, 8 GB, 32 partitions
B dataset	Task 1 on the integrated table	`local[*]`, 8 GB, 32 partitions
A/B training	Task 2 + Task 3 on each dataset	Task 3: `local[2]`, 4 GB, 8 partitions
Platform build	Ingestion + integration + Task 1	Platform notebooks' own settings

2 Comparison of the Two Approaches

Table 2: Summary of the comparison

Criterion	Measure	Approach A	Approach B
Implementation complexity	Code to build the dataset (lines)	463	240
	Cleaning checks written by the user	36	0
	Joins written by the user	5	0
Preprocessing complexity	Raw rows validated and deduplicated per run	9,554,778	0
	Time per dataset build (s)	31.5	20.5
Training time	Feature pipeline, Task 2 (s)	89.2	85.0
	Model fit (s)	27.5	27.1
	Task 3 total (s)	41.4	40.7
Reproducibility	Rows differing from the other approach	0	0
	Largest metric difference	1.1×10^{-5}	

2.1 Implementation and Preprocessing Complexity

Approach A needs almost twice as much code as Approach B: 463 versus 240 non-blank lines. Approach B does not need this extra code, because the platform (ingestion and integration, 676 lines) already did the work once in Weeks 1–3.

The extra code cleans each raw dataset in three steps and then joins it to the trips. It drops rows with an empty required field, drops rows with an invalid value (for example, a drop-off time before the pickup time or a negative fare), and removes duplicates by a key. Table 3 counts these steps. In total, Approach A re-implements 36 cleaning checks and five joins.

Table 3: Cleaning checks and joins that Approach A had to re-implement

Raw dataset	Required fields	Value checks	Duplicate key	Joined to each trip by
Taxi trips	5	7	`trip_id`	–
Weather	1	5	`observation_timestamp`	Pickup hour
Air quality	6	4	`observation_id`	Borough and pickup hour
Taxi zones	3	1	`location_id`	Pickup and drop-off zone
Total	15	17	4	5 joins

Line counts understate the effort. Some rules are not visible in the data and exist only in the platform code. Approach A produced the same rows as Approach B only after we found all of them:

- **How `trip_id` is built:** a SHA-256 hash of seven fields joined with `||`, with empty values written as `<null>`. Any other format gives different IDs.
- **Duplicates are removed before invalid rows.** Doing it the other way round keeps some duplicates that the platform rejects.
- **Time zone:** raw timestamps have no time zone, and the platform reads all of them as New York time, including the weather times that are actually UTC (Section 3). Approach A had to copy this behaviour to match.
- **Air-quality mapping:** county codes are mapped to boroughs, and several monitors are averaged per borough and hour.

2.2 Training Time

Table 4: Workflow time (median, seconds)

Workflow	Runs	Median (s)	Range (s)
Spark baseline	5	6.0	5.8–6.8
A dataset	5	31.5	29.6–31.9
B dataset	5	20.5	20.2–20.6
A training (Task 2 + Task 3)	3	130.6	127.1–134.5
B training (Task 2 + Task 3)	3	125.7	122.8–126.0
Platform build (ingestion 246.7, integration 28.9, Task 1 19.5)	3	294.9	288.9–296.9

Without the 6.0 s start-up cost, Approach A builds the dataset in 25.5 s and Approach B in 14.5 s. Half of Approach A's processing time (11.8 s of 23.1 s) is taxi validation and deduplication, which Approach B does not repeat. Model fitting takes the same time (27.5 s and 27.1 s), because the data is identical. Task 2 is 4.2 s slower on the Approach A dataset, which we attribute to its file layout: 96 small files instead of 22 larger ones.

The platform itself (ingestion and integration) costs 275.6 s once. It saves 11.0 s per dataset build, so it pays off after 26 builds (Figure 1). This favours Approach A: the platform also prepares data that this model does not use, such as air quality for every US state, and its cost is shared with the Week 2 analytical products.

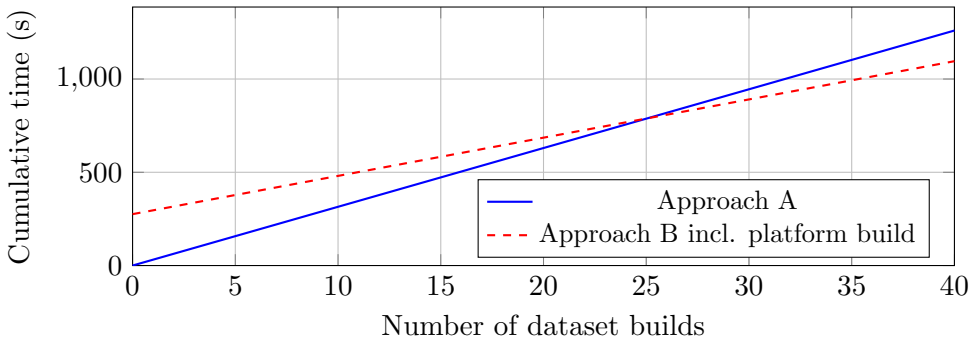


Figure 1: Cumulative dataset preparation time. The lines cross after 25.1 builds.

2.3 Model Results and Reproducibility

Table 5: Test metrics on each training dataset

Model	MAE (USD)	RMSE (USD)	R^2
Train-mean baseline	11.191	17.506	−0.002
Approach A, linear regression	6.99699	10.81009	0.61776
Approach B, linear regression	6.99700	10.81010	0.61776

The two training datasets are identical in every split (9,380,457 rows), and so are the fitted category mappings, imputation medians, and scaling ranges. The model metrics differ by at most 1.1×10^{-5}. L-BFGS sums gradients over partitions in every iteration, and the two datasets are partitioned differently, so rounding differs slightly after 50 iterations.

To test the platform itself, we rebuilt every platform table four times from the raw files and compared the business columns (all columns except _-prefixed audit metadata).

Table 6: Reproducibility of a full platform rebuild

Table	Rows	First vs last rebuild	Original vs rebuild
Taxi zones	265	identical	identical
Weather	8,783	identical	identical
Taxi trips	9,417,689	identical	identical
Air quality	7,927,367	identical	identical
Integrated taxi trips	9,417,689	identical	identical
Fare training dataset	9,380,457	identical	identical

All comparisons are identical, including against the original tables, which had gone through 35 `MERGE` and 37 `RESTORE` operations in Week 3. Reproducibility across team members was weaker: one member's local tables still contained the synthetic Week 3 update trips, so the first Validation and Test splits (April–June) consisted of shifted copies of Train trips.

3 Discussion

Which engineering decisions from Weeks 1–3 simplified machine learning? (1) Stable hash keys and deduplication at ingestion: every trip has one `trip_id`, and no model has to deduplicate again. (2) One integrated table at trip grain, with left joins and match-status columns: Task 1 selects features instead of writing joins. (3) Partitioning by pickup year and month: Task 1 reads only 2024. (4) Delta versions: the Task 3 manifest records the exact input versions, and time travel made the reproducibility test possible.

Which datasets contributed the most useful features? Table 7 groups the 561 model features by source dataset and lists the largest coefficient in each group. Coefficients are in US dollars per category or per interquartile range, so they show size, not a full importance ranking.

Table 7: Model features by source dataset

Dataset	Features	Largest |coef.| (USD)	Strongest feature
Taxi zone lookup	515	14.73	Drop-off zone and borough
Taxi trips	24	14.68	Rate code (e.g. JFK flat fare)
Air quality	2	6.40	Missing-value indicator
Weather	20	0.42	Weather condition code

The zone lookup and the trip's own fields carry the model. Weather changes the predicted fare by less than \$0.50, although this may be underestimated because weather is joined with a 4–5 hour offset (see below). The air-quality indicator has a large coefficient, but PM2.5 is missing for 90% of trips, so the indicator most likely marks where monitors exist rather than air quality itself. Two expected signals are lost in preprocessing: `trip_distance` has a coefficient near zero, probably because of extreme distances (above 10,000 miles), and three features (passenger count, precipitation, PM2.5) have an interquartile range of 0 and are scaled to 0.

Which parts of the workflow became reusable? The integrated Delta table; the Spark session builder; the shared `week4` module with the feature pipeline, regressor, and `evaluate_split()`; the Task 2 and Task 3 notebooks, which now run on any compatible dataset through `WEEK4_VARIANT`; and the timing harness. Before Task 4, the feature pipeline and evaluation existed only inside notebooks and could not be imported. Approach A could reuse nothing from Weeks 1–3.

How did the integrated platform reduce preprocessing before feature engineering? Approach B starts after validation, deduplication, and integration are finished. It skips 36 checks, five joins, and the validation and deduplication of 9.55 million raw trips (137,089 rejected). All consumers also share the same rejected rows and the same join results, which per-experiment preprocessing cannot guarantee.

How would the ML pipeline change if additional datasets were introduced? With the platform, a new dataset needs an ingestion configuration with validation rules and one new join in the integration step. The ML pipeline then changes in two places only: Task 1 selects the new columns, and `week4/features.py` adds them to a feature group. Task 3 and the evaluation do not change. With Approach A, every experiment must implement the new loading, cleaning, and join code again. A dataset at a different grain, for example events by location, would also need a new join key in the integrated table.

What improvements to the platform would most improve future machine learning applications?

- **Explicit time zones.** The weather data comes from NOAA's Integrated Surface Database, which records times in UTC, but the platform builds the timestamp as New York time and never converts it. The data confirms this: the coldest hour in January–March is 11:00 and the warmest is 20:00, which matches 06:00 and 15:00 in New York only after subtracting five hours. Every trip is therefore joined with weather from 4–5 hours earlier. The same mistake turns 02:00 UTC on 10 March 2024 into 02:00 New York time, an hour that does not exist because clocks jump from 02:00 to 03:00 when daylight saving time starts. It becomes 03:00 and collides with the next row.
- **Deterministic deduplication.** When two duplicates come from the same file, the platform does not define which one survives. In the collision above it kept the same row in all our runs,

but nothing guarantees this. Adding the row position in the file as a second sort key would fix it.

- **Feature-quality checks.** The platform profiles missing values, but nothing checks whether a feature survives scaling. A check for zero spread would have caught the three zeroed features before training.
- **Compacted ML tables.** Writing training tables with fewer, larger files would remove the small-file overhead seen in Task 2.

Limitations. All results come from one laptop, three months of Taxi data, and warm file caches. They show relative costs; absolute times on a cluster or with more history would differ.